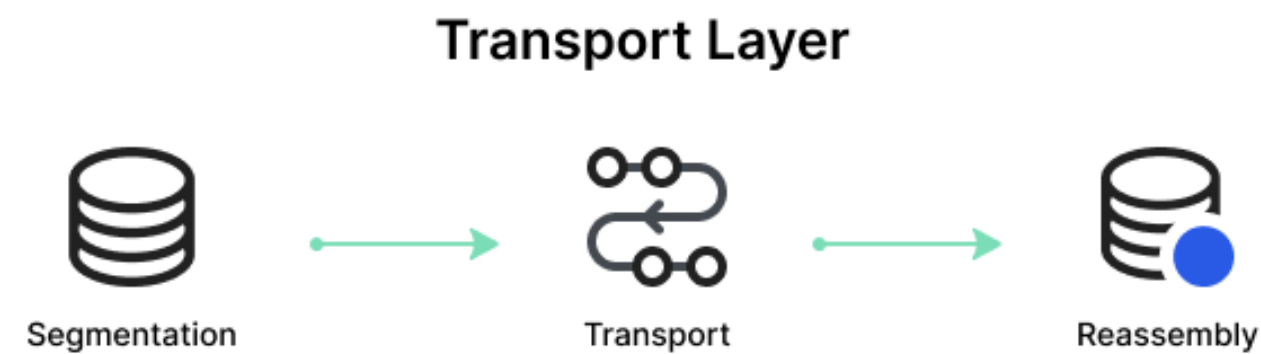


TCP Congestion Control Algorithms

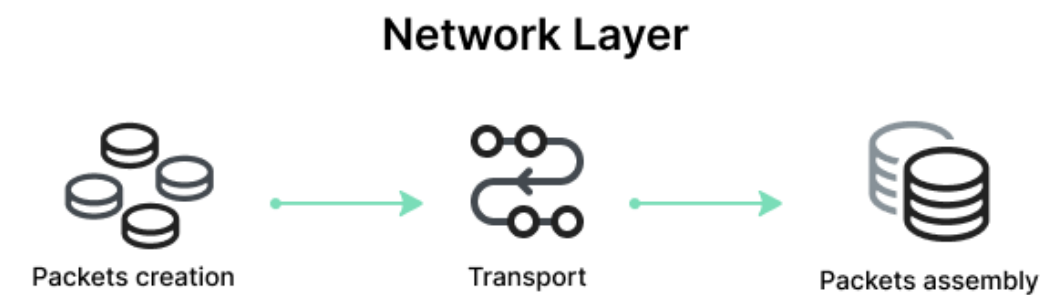


Join the lecture online on *your* dashboard

TCP recap

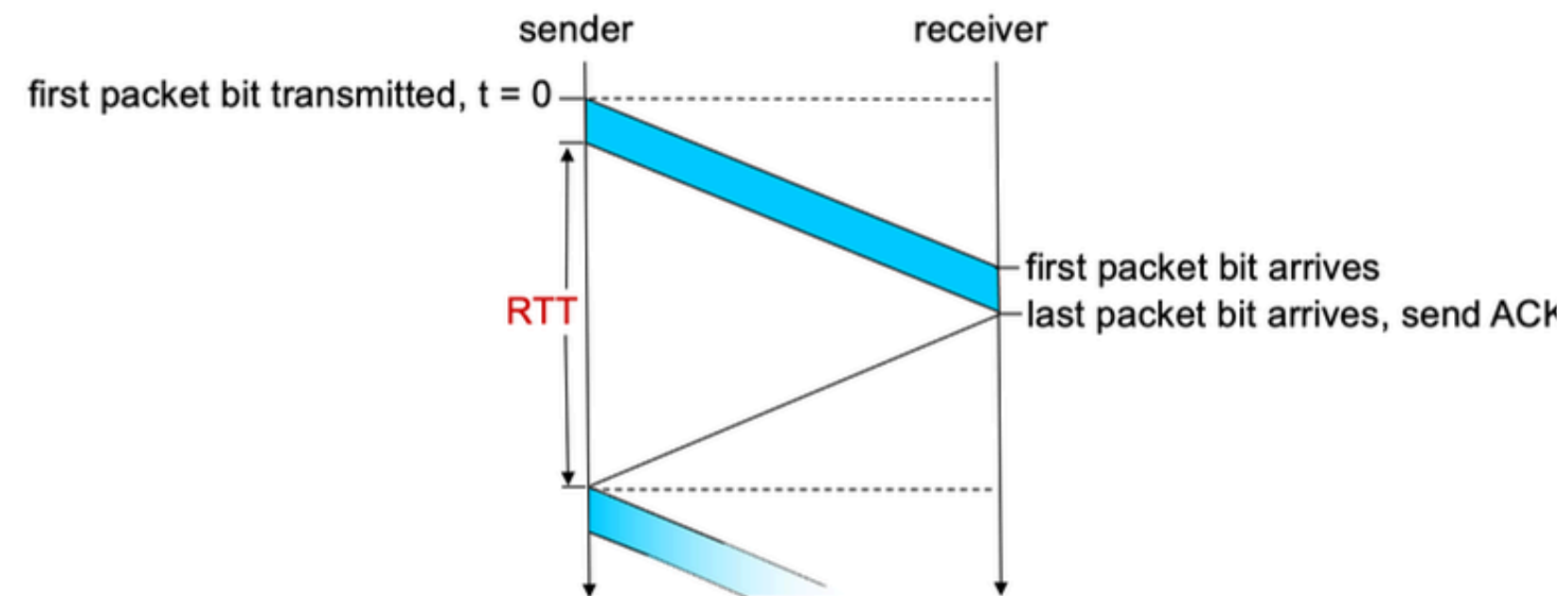


**transport layer: communication
between processes**
segments oriented

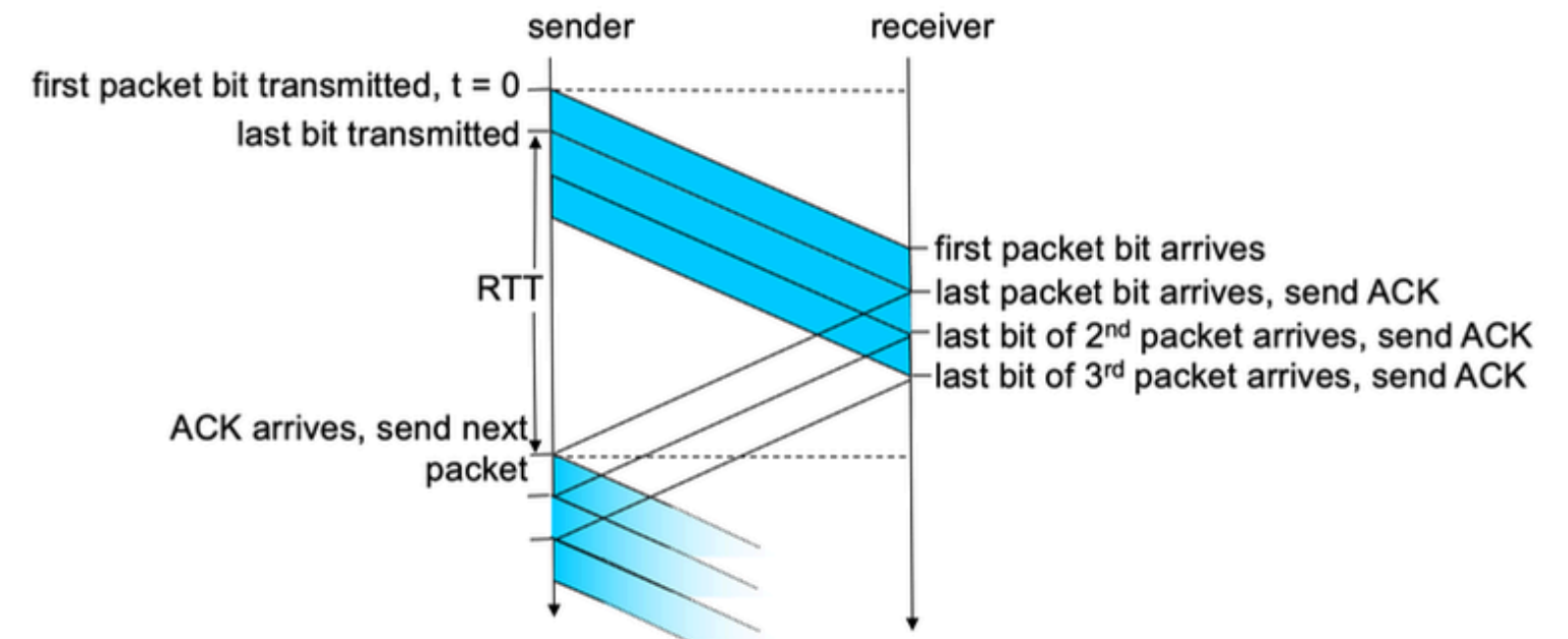


**network layer: communication between
hosts**

TCP recap

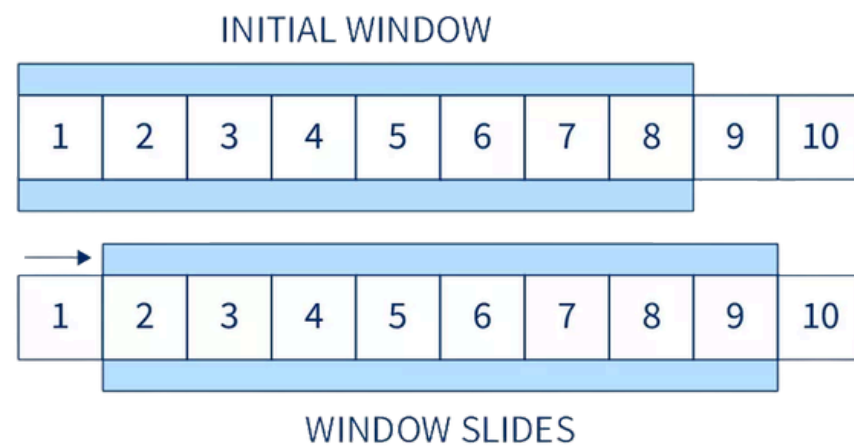


**Packet delivery time = Transmission
time + Propagation delay**



Pipelining: increased utilization

Sliding Window Protocol



- TCP uses a "sliding window protocol" to manage flow control.
- The receiver tells the sender how much data it can receive at a time (called the window size).
- As the receiver processes data and acknowledges packets, the sender "slides" the window forward and sends more.



**TCP flow control helps for a single client!
What if the network is crowded?**

Let's look at the cars analogy...



Only Mater (client) wants to move slowly



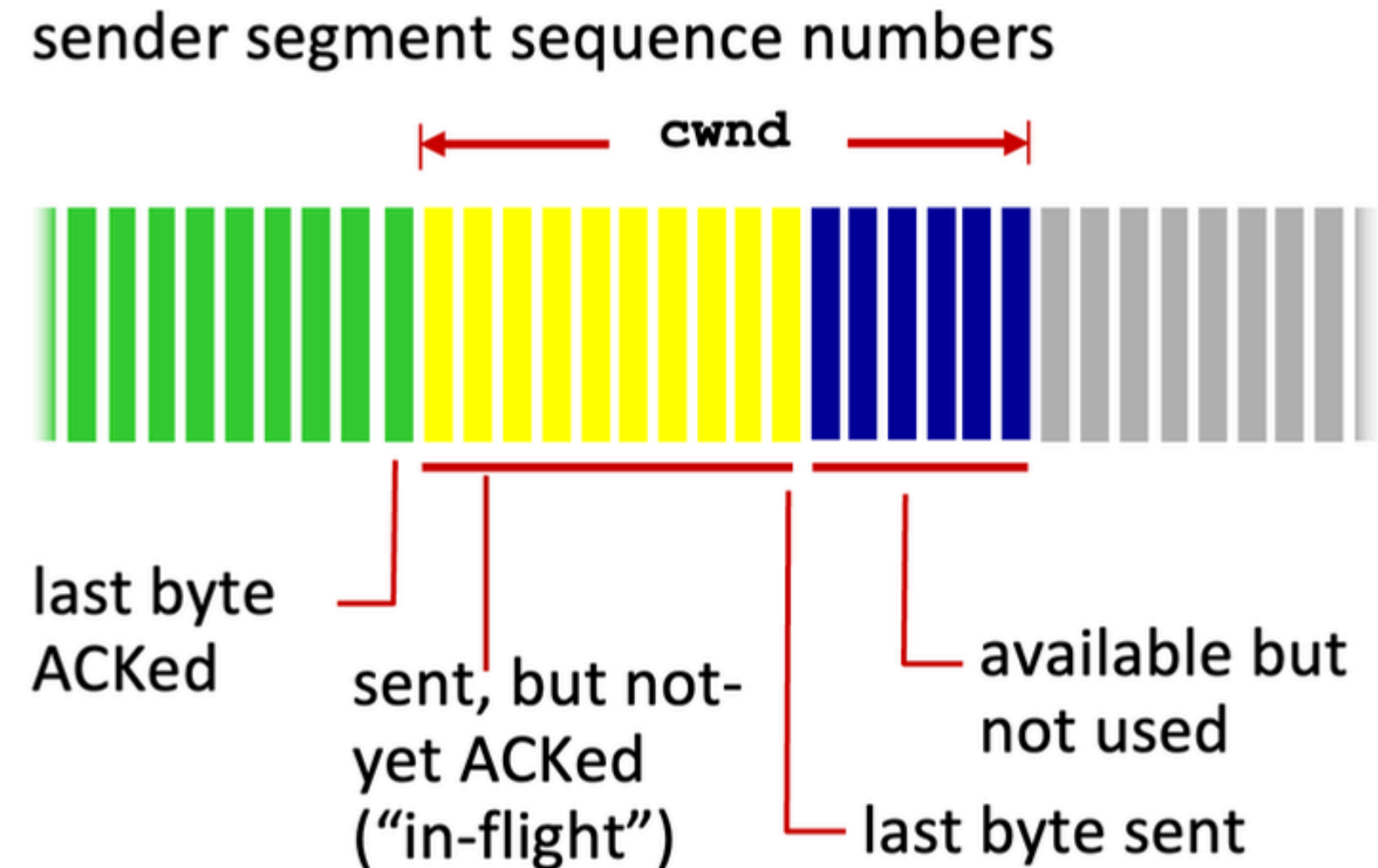
The Traffic Jam (Congestion)
All senders have to slow down

Congestion Window

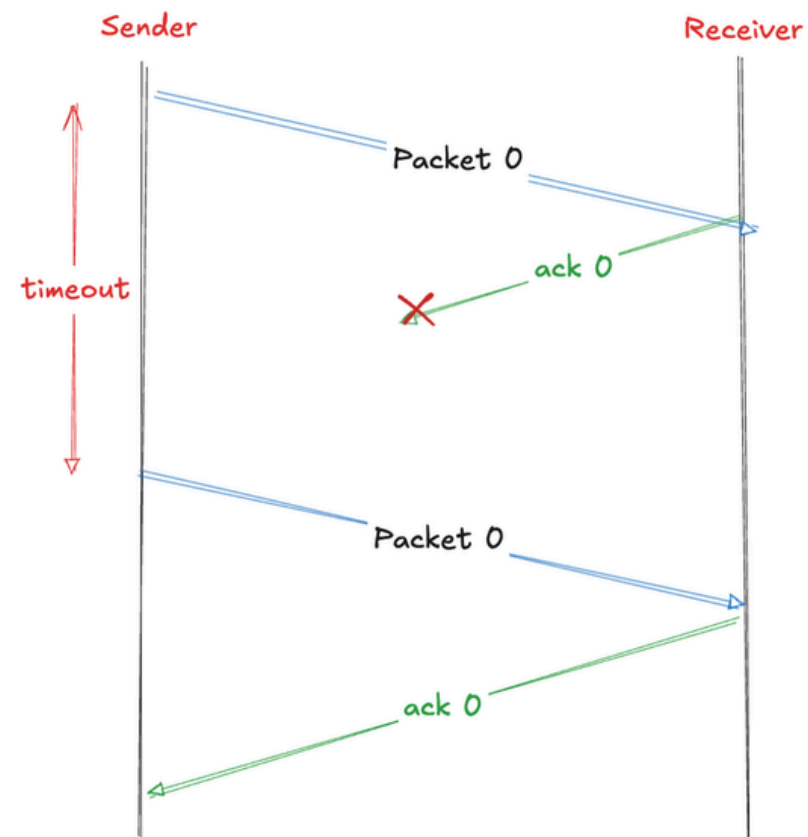
Congestion window size (cwnd)

Sender-side limit on the amount of data the sender can have in flight before receiving an acknowledgment (ACK) from the client.

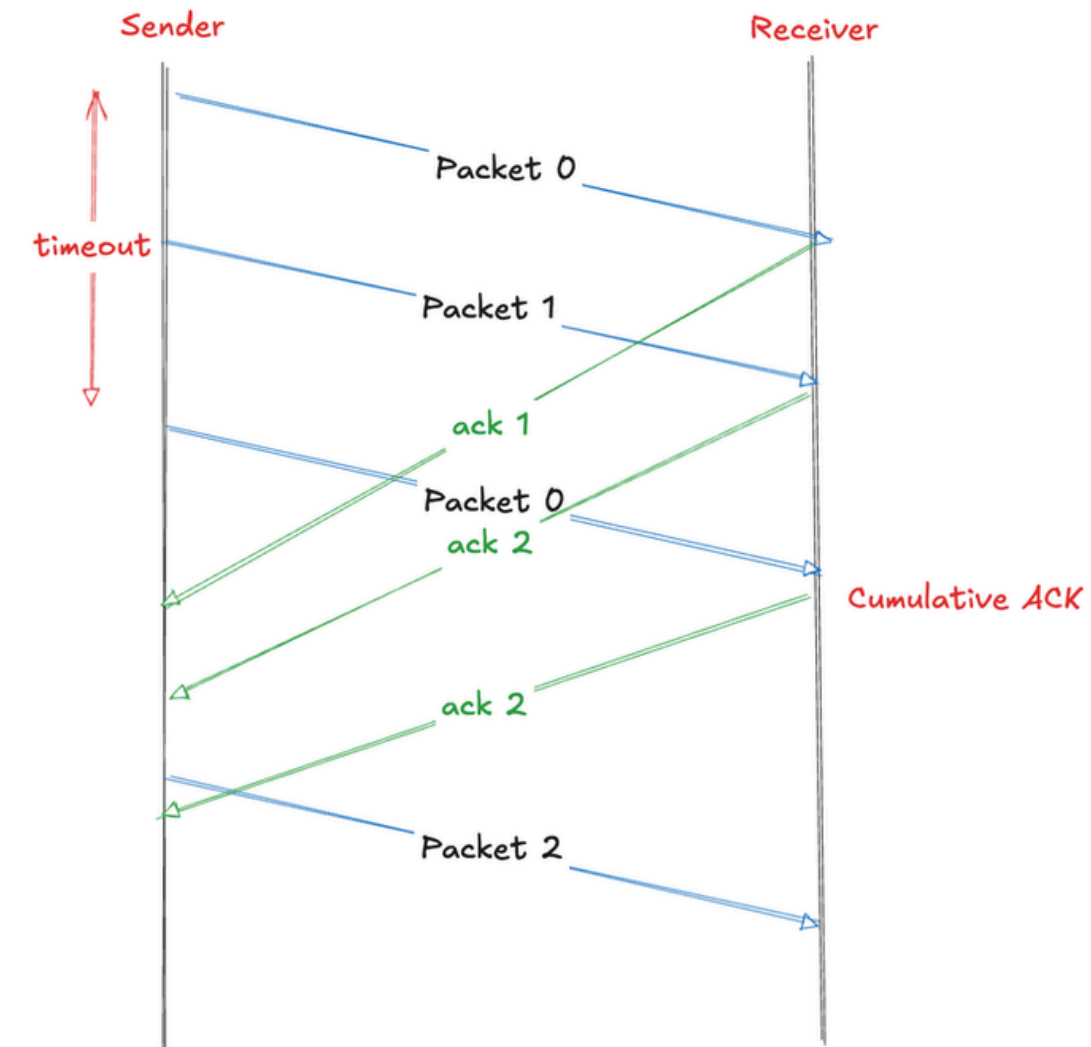
For new connections it is 10 by default in most operating systems



Two special cases of TCP timeouts

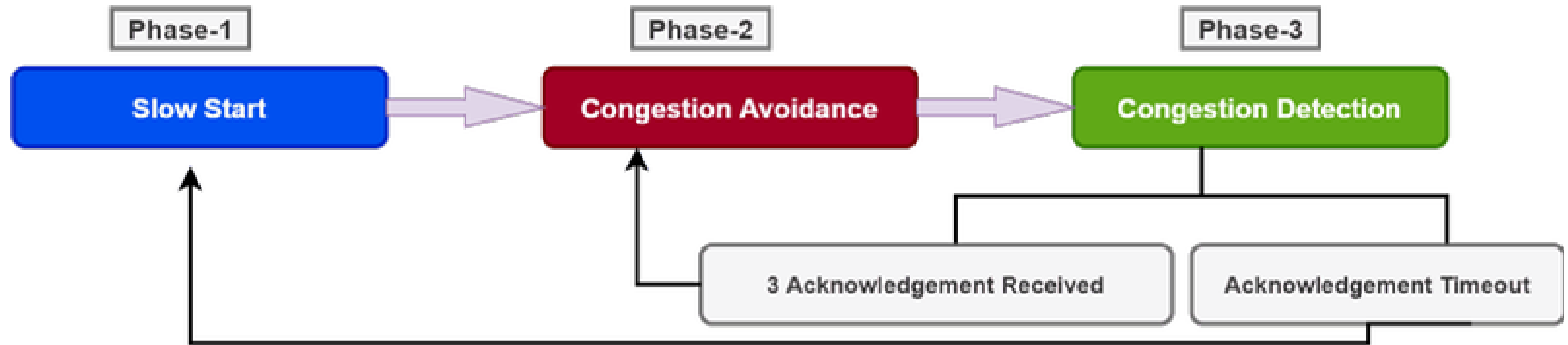


lost ACK scenario



premature timeout

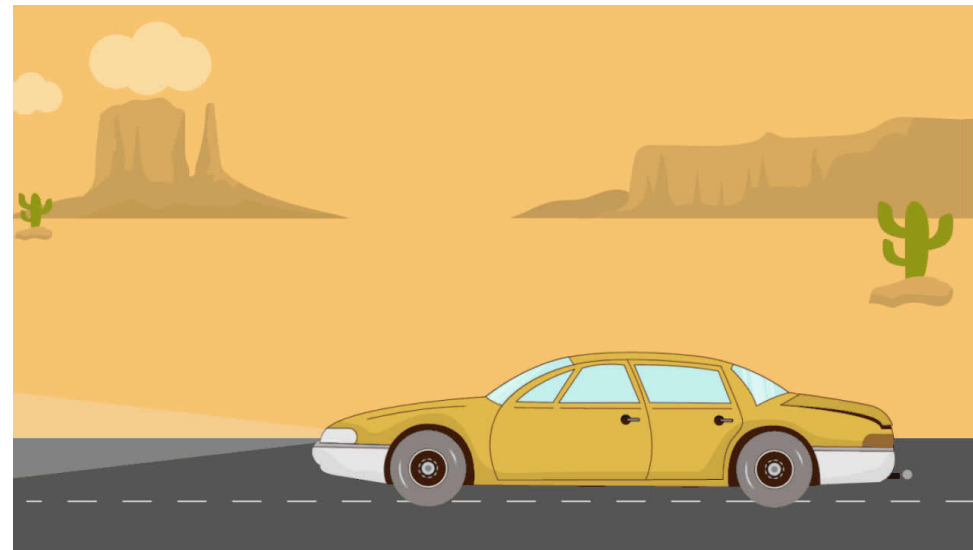
TCP Congestion Control



- TCP is a polite driver.
- Its mission: Get your data to its destination as fast as possible, but without causing a traffic jam for everyone else.

Slow Start

1



slow start, send just
a single car on the
road

2



Next time send
two cars

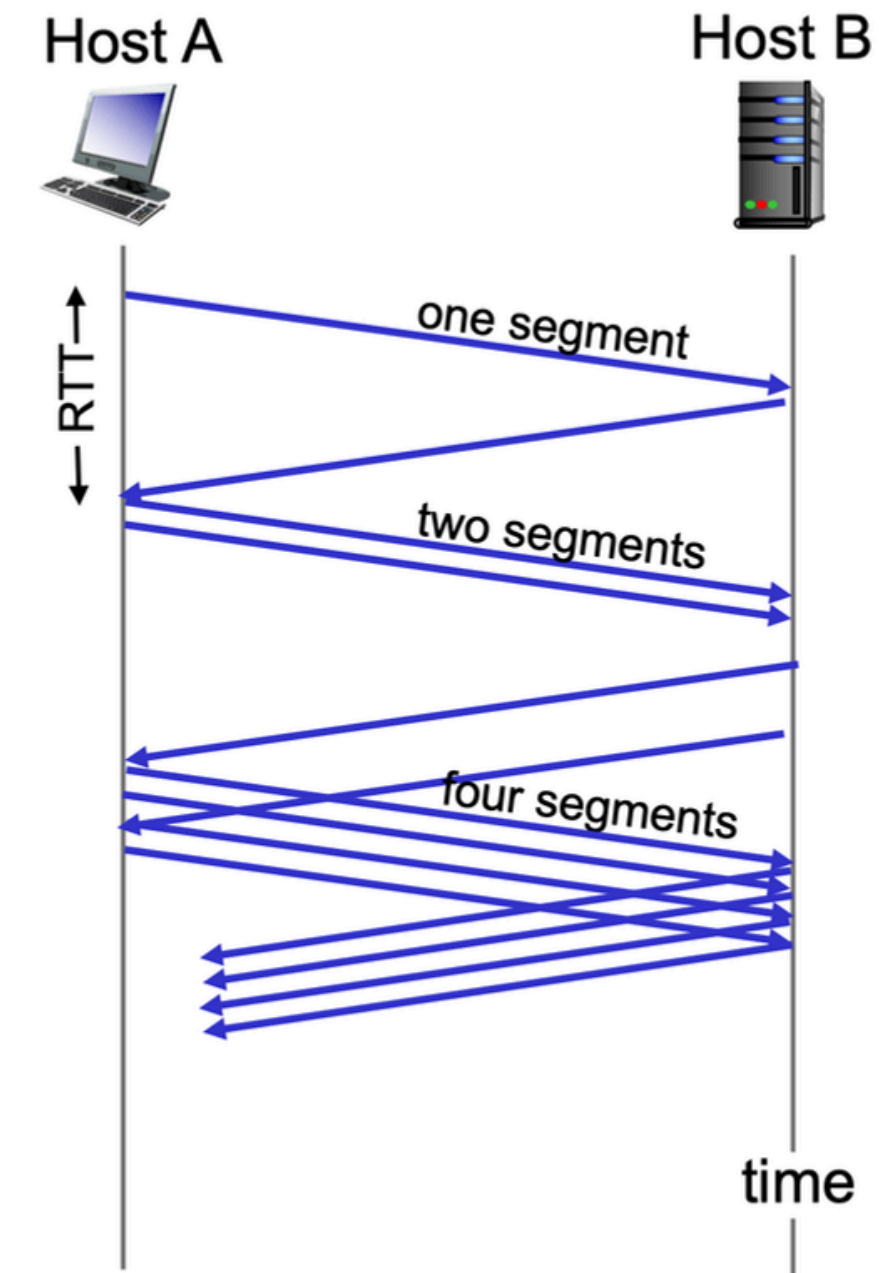
4



Then send four cars

Slow Start

- When the connection is established, it starts slowly in a phase called Slow Start.
- Initially, it sends a small number of send.
- If those packets are acknowledged, it doubles the number sent.
- This doubling helps to find an optimal number of segments to send



When Does Slow Start End?

Slow Start continues until:

- congestion window reaches a threshold called slow start threshold (sssthresh),

$$\text{sssthresh} = \text{cwnd}/2$$

half of the value of the congestion window value when congestion was detected.

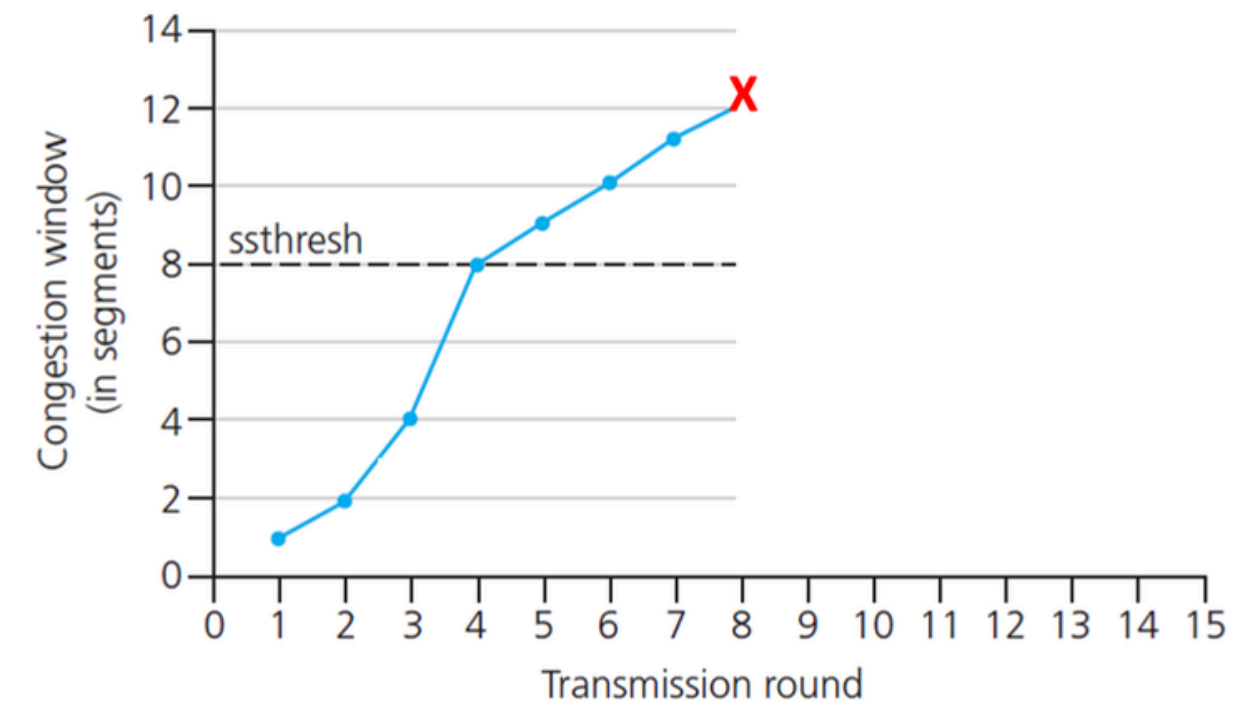
It switches to **Congestion Avoidance**.

Instead of doubling, it now adds just one extra packet at a time.

This is a gentle, linear increase.

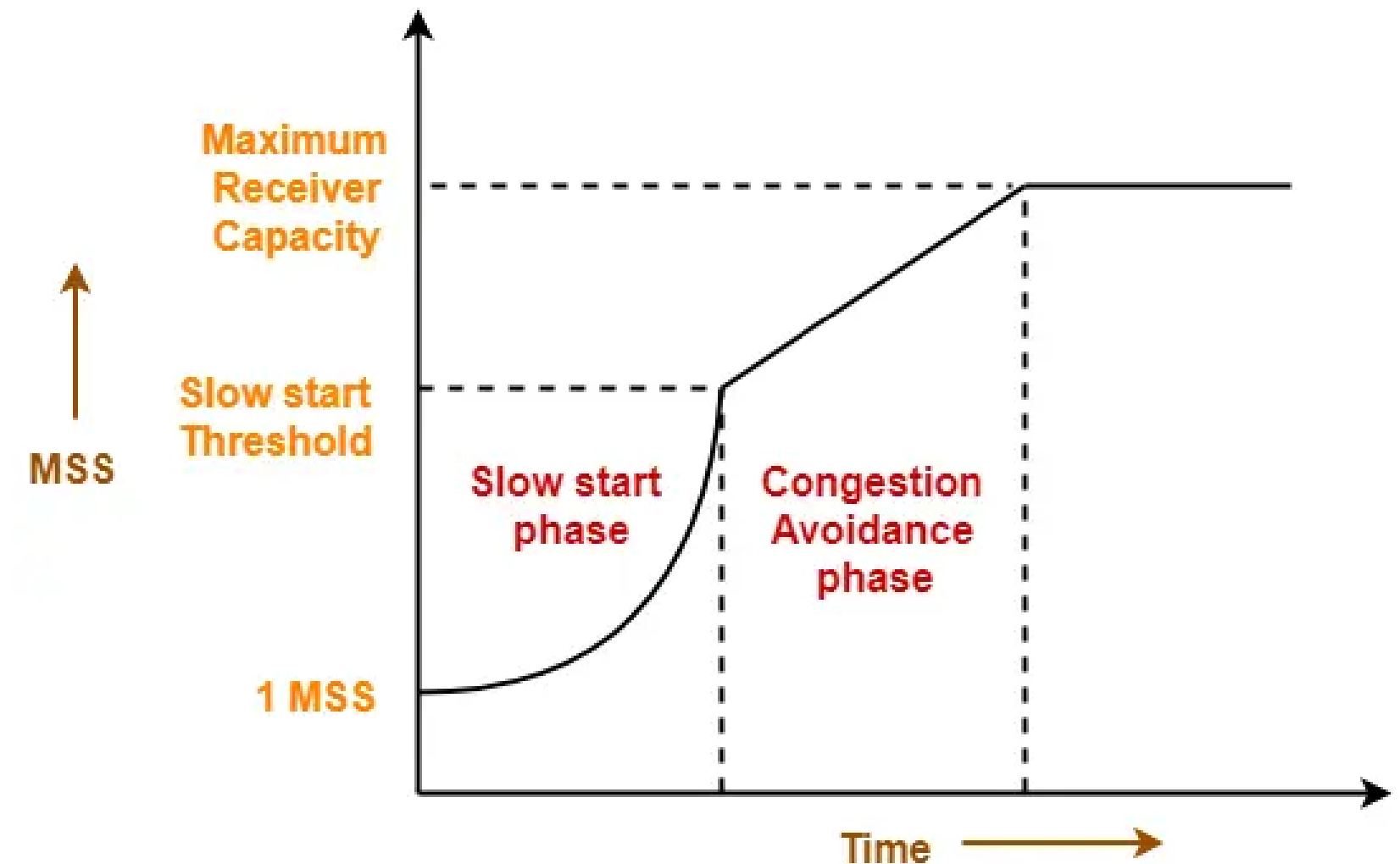
OR

- *Packet loss occurs (timeout or 3 duplicate ACKs)*



Congestion Avoidance

MSS (Maximum Segment Size) is the largest amount of data, in bytes, that a device can send in the payload of a single TCP segment



AIMD (Additive Increase, Multiplicative Decrease)

approach: senders can increase sending rate until packet loss (congestion) occurs, then decrease sending rate on loss event

Additive Increase

- Every RTT (Round Trip Time) with no packet loss:
Increase cwnd linearly by 1 MSS (Maximum Segment Size).

Multiplicative Decrease

- sending rate: Cut in half on loss detected by three duplicate ACKs
- Cut to 1 MSS (maximum segment size) when loss detected by timeout

The TCP Sawtooth Pattern



AIMD operates by having the sender increase its transmission rate (Additive Increase) to probe for bandwidth. Upon packet loss, indicating congestion, it halves the rate (Multiplicative Decrease), creating a sawtooth wave pattern.

Fast Retransmit and Fast Recovery in TCP

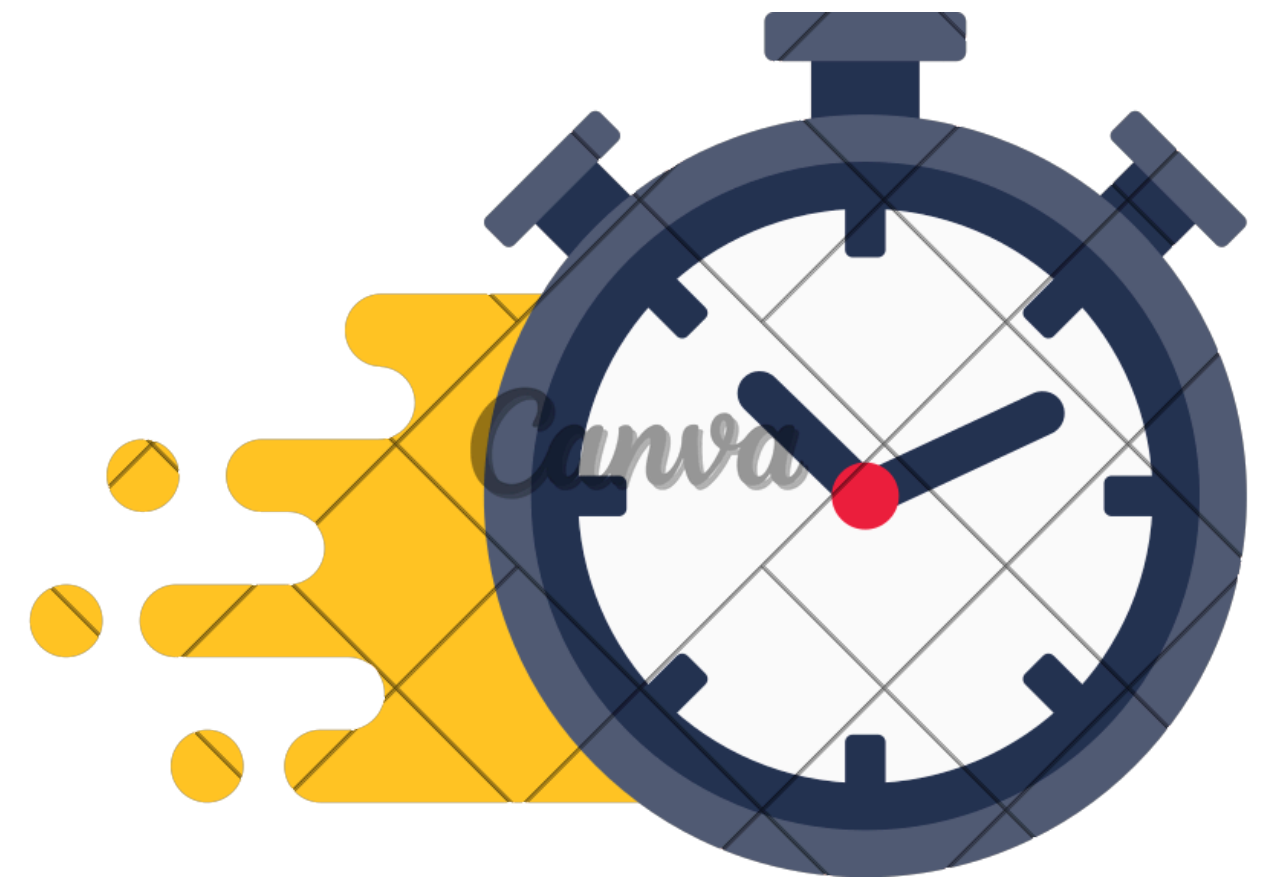
Why Are They Needed?

TCP is designed to detect packet loss either through:

- A timeout (slow and inefficient), or
- Multiple duplicate ACKs (faster mechanism)

To speed up recovery and avoid unnecessary delays, TCP uses:

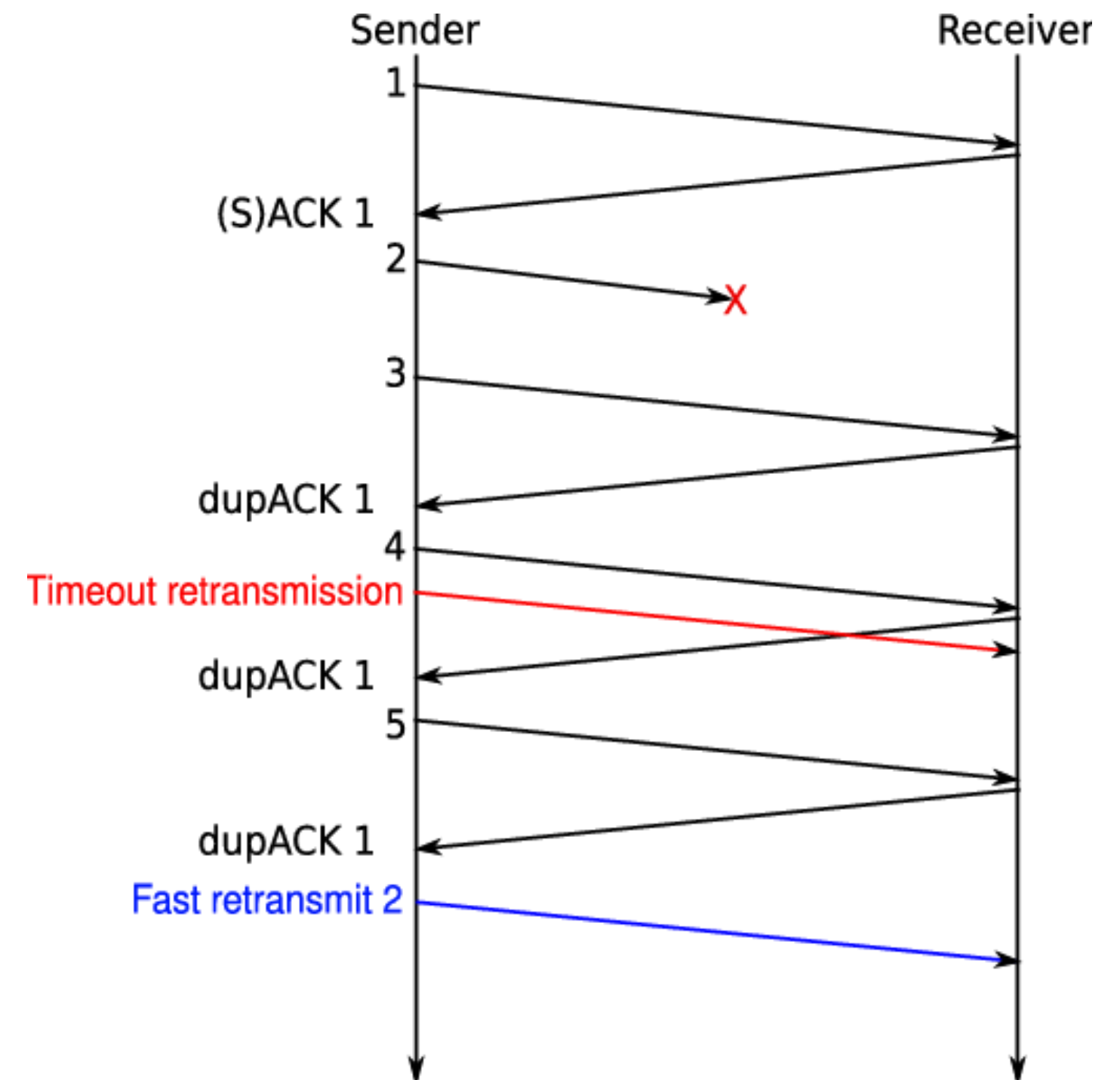
1.  Fast Retransmit
2.  Fast Recovery



Fast Retransmit

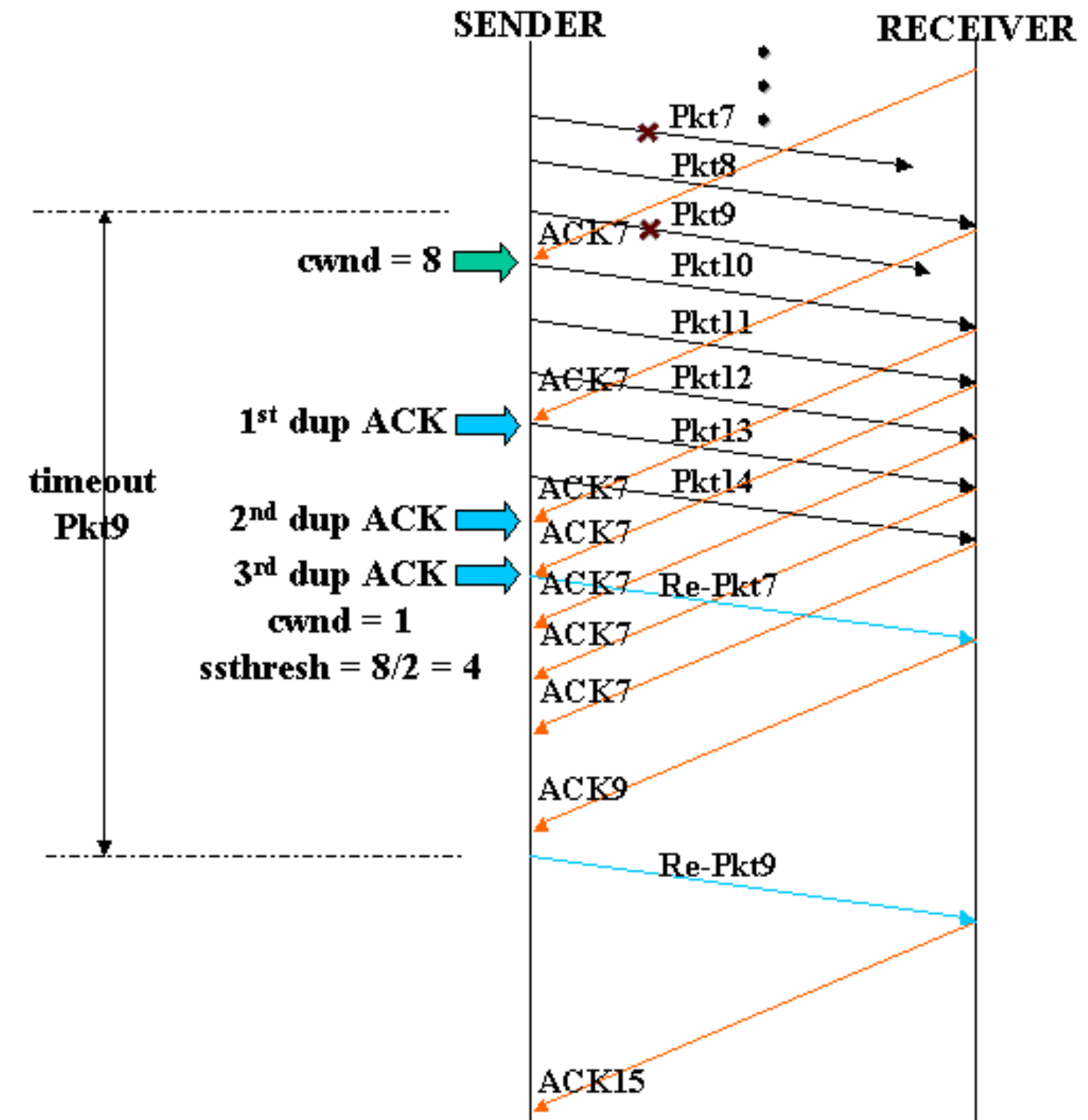
- If the sender receives three duplicate ACKs (e.g., ACK 5, ACK 5, ACK 5),
- It assumes a packet was lost, likely the next in sequence,
- It immediately retransmits that missing packet.

Faster than waiting for a timer to expire!



Newton School
of Technology

Instead of going back to Slow Start, TCP enters Congestion Avoidance directly, allowing smoother performance.



Step-by-Step example for fast recovery

Assume:

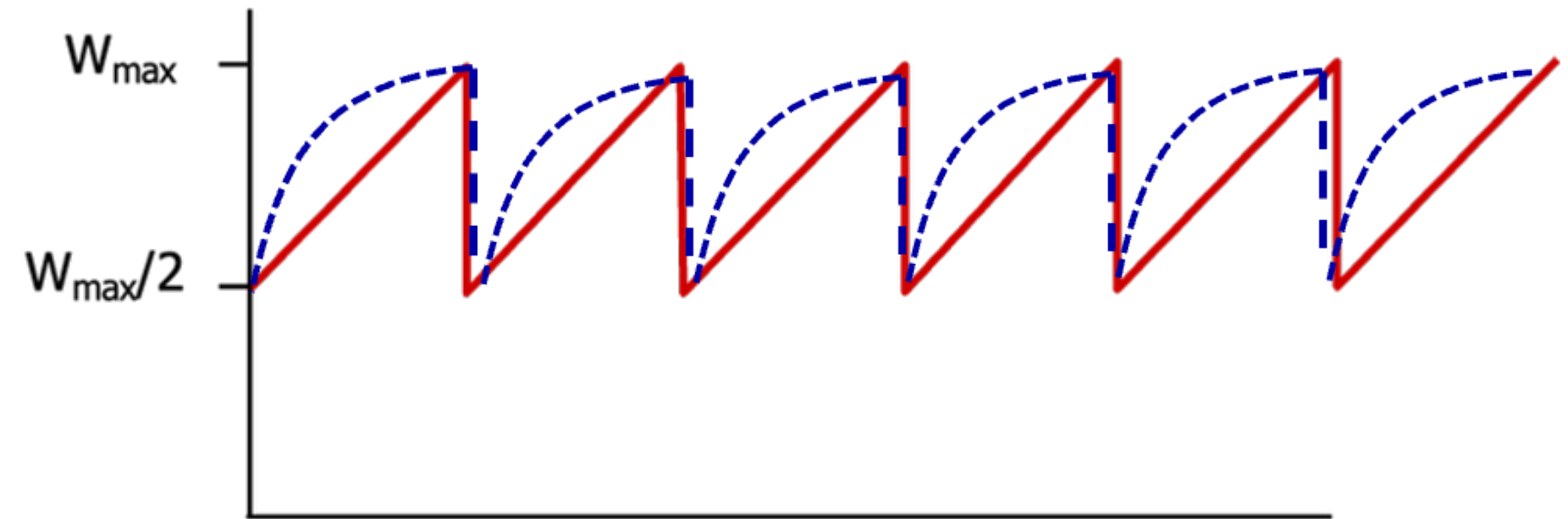
- ssthresh = 8 MSS, cwnd = 16 MSS, TCP is in Congestion Avoidance in RTT 3

Event	cwnd (in MSS)	Explanation
RTT 1 (no loss)	17	Additive increase (cwnd += 1)
RTT 2 (no loss)	18	cwnd += 1
RTT 3 (tcp timeout)	9	Multiplicative decrease (cwnd / 2)
RTT 4 (no loss)	10	Resume additive increase

TCP CUBIC

Is there a better way than AIMD to “probe” for usable bandwidth?

increase W as a function of the cube of the distance between current time and time when TCP window size will reach $W_{\max}$



$W_{\max}$ sending rate at which
congestion loss was
detected

— classic TCP
- - - TCP CUBIC - higher
throughput in this
example

TCP CUBIC default in Linux, most popular TCP for Web servers!

Please fill the feedback form.

**Thanks
for
watching!**